```
namespace messaging
{
    class receiver
    {
        queue q;
    public:
        operator sender()
        {
            return sender(&q);
        }

        dispatcher wait()
        {
            return dispatcher(&q);
        }
    };
}
```

A receiver owns the queue

Allow implicit conversion to a sender that references the queue

Waiting for a queue creates a dispatcher

Whereas a `sender` just references a message queue, a `receiver` owns it. You can obtain a `sender` that references the queue by using the implicit conversion. The complexity of doing the message dispatch starts with a call to `wait()`. This creates a `dispatcher` object that references the queue from the `receiver`. The `dispatcher` class is shown in the next listing; as you can see, the work is done in the *destructor*. In this case, that work consists of waiting for a message and dispatching it.

Listing C.4. The dispatcher class

```
namespace messaging
{
    class close_queue
    {};

    class dispatcher
    {
        queue* q;
        bool chained;

        dispatcher(dispatcher const&)=delete;
        dispatcher& operator=(dispatcher const&)=delete;
```

The message for closing the queue

dispatcher instances cannot be copied